Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Основы системного программирования»

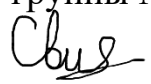
ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Процессы, потоки и средства межпроцессного взаимодействия»

Вариант 16

Выполнил:

Свиржевский Алексей Денисович, студент группы N3252



(подпись)

Проверил:

Грозов Владимир Андреевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание 3
Задание 3
1. ОПИСАНИЕ ЛОГИКИ РАБОТЫ 4
2. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ 6
ВЫВОД 11
Приложение 12

ЗАДАНИЕ

Цель работы: изучение механизмов создания сетевых приложений (клиент-сервер), освоение работы с процессами (многозадачность), сигналами,

демонизацией и средствами межпроцессного взаимодействия (разделяемая память).

Задание по варианту плагина (Вариант 16):

1. Тип многозадачности: многопроцессность.
2. Транспортный протокол: UDP.
3. Прикладной протокол:
    - Запрос: от 2 до 256 байтов, первый байт кодирует длину данных, остальные – сами данные.
    - Ответ: строка, содержащая двоичное представление каждого байта данных (байты разделены пробелами), завершающаяся символом LF. При ошибке возвращается строка ERROR N\n.

1. ОПИСАНИЕ АРХИТЕКТУРЫ

Разработанный программный комплекс состоит из двух утилит: lab3asdN3252_server и lab3asdN3252_client

1. **Серверная часть (server.c):**
    - Демонизация: поддерживает запуск в фоновом режиме (опция -d) через двойной fork(), смену корневого каталога и перенаправление

стандартных потоков ввода-вывода в /dev/null (с сохранением stderr для вывода статистики).

- Многозадачность: основной процесс выполняет recvfrom, блокируясь в ожидании UDP-датаграмм. При получении запроса вызывается fork(). Дочерний процесс валидирует пакет, формирует ответ (sendto) и завершается.
- Предотвращение зомби-процессов: установлен асинхронный обработчик SIGCHLD, который в цикле вызывает waitpid(-1, NULL, WNOHANG), корректно освобождая ресурсы таблицы процессов ОС.
- Межпроцессное взаимодействие (IPC): статистика сервера (счетчики успешных запросов и ошибок) размещена в разделяемой памяти с помощью системного вызова mmap(MAP_SHARED| MAP_ANONYMOUS). Для избежания состояний гонки между параллельными дочерними процессами счетчики обновляются через атомарные операции atomic_fetch_add.
- Обработка сигналов: обработчики зарегистрированы через безопасный интерфейс sigaction. По сигналу SIGUSR1 сервер пишет текущую статистику в лог и stderr. Сигналы SIGINT, SIGTERM, SIGQUIT инициируют штатную остановку.

2. **Клиентская часть (client.c)**
    - Парсит введенную пользователем Hex-строку (с учетом префиксов 0x) в сырой бинарный буфер, формирует UDP-датаграмму и отправляет серверу по заданному IP и порту. Установлен таймаут на получение ответа (SO_RCVTIMEO), чтобы клиент не зависал при потере UDP-пакета или отсутствии сервера.

2. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Сборка проекта и тестирование осуществляются с помощью утилиты make.

Содержимое Makefile:

```
SHELL := /bin/bash
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE

SERVER = lab3asdN3252_server
CLIENT = lab3asdN3252_client
```

```
# Флаги Valgrind (отслеживание форков и запись логов по PID)
VFLAGS = valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes
--trace-children=yes --log-file=valgrind_%p.txt

all: $(SERVER) $(CLIENT)

$(SERVER): server.c
	$(CC) $(CFLAGS) -o $@ $^

$(CLIENT): client.c
	$(CC) $(CFLAGS) -o $@ $^

clean:
	rm -f $(SERVER) $(CLIENT) valgrind*.txt
	rm -f /tmp/lab3.log

test: all
	@echo -e "\n 1. ЗАПУСК СЕРВЕРА (Демон, порт 8080) "
	rm -f /tmp/lab3.log
	./$(SERVER) -d -p 8080 -a 127.0.0.1
	@sleep 1
	@echo -e "\n 2. ОТПРАВКА КОРРЕКТНОГО ЗАПРОСА "
	./$(CLIENT) 127.0.0.1 8080 0x02AABB
	@echo -e "\n 3. ТЕСТ: СТРОГОСТЬ ПРОТОКОЛА "
	@echo "[Тест 3.1] Пакет короче 2 байт (только длина):"
	./$(CLIENT) 127.0.0.1 8080 0x05 || true
	@echo "[Тест 3.2] Пакет слишком длинный (указано 2, по факту 4):"
	./$(CLIENT) 127.0.0.1 8080 0x02AABBCCDD || true
	@echo "[Тест 3.3] Пакет слишком короткий (указано 5, по факту 2):"
	./$(CLIENT) 127.0.0.1 8080 0x05AABB || true
	@echo -e "\n 4. ТЕСТ: ИСТИННАЯ МНОГОЗАДАЧНОСТЬ (fork) "
	@echo "Запускаем второй сервер на порту 8081 с задержкой обработки 3
секунды..."
	./$(SERVER) -d -p 8081 -a 127.0.0.1 -w 3
	@sleep 1
	@echo "Отправляем долгий запрос (сервер уснет на 3 сек)..."
	./$(CLIENT) 127.0.0.1 8081 0x01FF &
```

```
	@sleep 0.5
	@echo "Сразу отправляем быстрый запрос..."
	./$(CLIENT) 127.0.0.1 8081 0x01AA
	@echo "Ждем завершения первого запроса..."
	@wait
	@echo -e "\n 5. ТЕСТ: ПРИОРИТЕТ ПЕРЕМЕННЫХ ОКРУЖЕНИЯ "
	@echo "Задаем lab3PORT=9000, но передаем опцию -p 8082."
	@echo "Ожидание: сервер должен слушать 8082 (опция приоритетнее)."
	lab3PORT=9000 ./$(SERVER) -d -p 8082 -a 127.0.0.1
	@sleep 1
	./$(CLIENT) 127.0.0.1 8082 0x01BB
	@echo -e "\n 6. ВЫЗОВ СТАТИСТИКИ (SIGUSR1) "
	killall -SIGUSR1 $(SERVER) || true
	@sleep 1
	@echo -e "\n 7. ОСТАНОВКА ВСЕХ СЕРВЕРОВ (SIGTERM) "
	killall -SIGTERM $(SERVER) || true
	@sleep 1
	@echo -e "\n 8. ИТОГОВЫЙ ЛОГ ФАЙЛ "
	@cat /tmp/lab3.log

valgrind_test: all
	@echo -e "\n  ТЕСТИРОВАНИЕ УТЕЧЕК ПАМЯТИ (VALGRIND)  "
	@rm -f valgrind*.txt /tmp/lab3.log
	@echo "[+] Запуск сервера под Valgrind..."
	$(VFLAGS) ./$(SERVER) -d -p 8080 -a 127.0.0.1
	@sleep 2
	@echo  "[+]  Отправка  валидного  запроса  клиентом  (клиент  тоже  под
Valgrind)..."
	valgrind --leak-check=full --log-file=valgrind_client_%p.txt ./$(CLIENT)
127.0.0.1 8080 0x02AABB
	@echo "[+] Отправка невалидного запроса..."
	./$(CLIENT) 127.0.0.1 8080 0x05 || true
	@sleep 1
	@echo "[+] Остановка сервера для выгрузки отчетов Valgrind..."
	pkill -SIGTERM -f "[l]ab3asdN3252_server" || true
	@sleep 2
```

```
	@echo -e "\n  СВОДКА ОШИБОК И УТЕЧЕК VALGRIND  "
	@grep -H -E "ERROR SUMMARY|definitely lost|possibly lost" valgrind*.txt
|| echo "Логи не найдены или пусты"
	@echo -e "\nПодробные отчеты сохранены в файлах valgrind_<PID>.txt"

archive: clean
	cd .. && tar -czvf $(SERVER).tar.gz lab3asdN3252/
	@echo -e "\n Архив готов! "
```

make test (рис. 1 и 2):

```
 1. ЗАПУСК СЕРВЕРА (Демон, порт 8080)
rm -f /tmp/lab3.log
./lab3asdN3252_server -d -p 8080 -a 127.0.0.1

 2. ОТПРАВКА КОРРЕКТНОГО ЗАПРОСА
./lab3asdN3252_client 127.0.0.1 8080 0x02AABB
Ответ сервера: 10101010 10111011

 3. СТРЕСС-ТЕСТ: СТРОГОСТЬ ПРОТОКОЛА
[Тест 3.1] Пакет короче 2 байт (только длина):
./lab3asdN3252_client 127.0.0.1 8080 0x05 || true
Ответ сервера: ERROR 1
[Тест 3.2] Пакет слишком длинный (указано 2, по факту 4):
./lab3asdN3252_client 127.0.0.1 8080 0x02AABBCCDD || true
Ответ сервера: ERROR 2
[Тест 3.3] Пакет слишком короткий (указано 5, по факту 2):
./lab3asdN3252_client 127.0.0.1 8080 0x05AABB || true
Ответ сервера: ERROR 2

 4. ТЕСТ: ИСТИННАЯ МНОГОЗАДАЧНОСТЬ (fork)
Запускаем второй сервер на порту 8081 с задержкой обработки 3 секунды...
./lab3asdN3252_server -d -p 8081 -a 127.0.0.1 -w 3
Отправляем долгий запрос (сервер уснет на 3 сек)...
./lab3asdN3252_client 127.0.0.1 8081 0x01FF &
Сразу отправляем быстрый запрос (должен обработаться МГНОВЕННО!)...
./lab3asdN3252_client 127.0.0.1 8081 0x01AA
Ответ сервера: 11111111
Ответ сервера: 10101010
Ждем завершения первого (долгого) запроса...

 5. ТЕСТ: ПРИОРИТЕТ ПЕРЕМЕННЫХ ОКРУЖЕНИЯ
Задаем lab3PORT=9000, но передаем опцию -p 8082.
Ожидание: сервер должен слушать 8082 (опция приоритетнее).
lab3PORT=9000 ./lab3asdN3252_server -d -p 8082 -a 127.0.0.1
./lab3asdN3252_client 127.0.0.1 8082 0x01BB
Ответ сервера: 10111011
```

Рисунок 1

```
 6. ВЫЗОВ СТАТИСТИКИ (SIGUSR1)
killall -SIGUSR1 lab3asdN3252_server
СТАТИСТИКА: Время работы: 7 сек | Успешных: 1 | Ошибок: 3
СТАТИСТИКА: Время работы: 5 сек | Успешных: 2 | Ошибок: 0
СТАТИСТИКА: Время работы: 1 сек | Успешных: 1 | Ошибок: 0

 7. ОСТАНОВКА ВСЕХ СЕРВЕРОВ (SIGTERM)
killall -SIGTERM lab3asdN3252_server

 8. ИТОГОВЫЙ ЛОГ ФАЙЛ
[20.05.26 21:49:56] Сервер запущен (UDP)
[20.05.26 21:49:57] Новый запрос от 127.0.0.1:45804 (размер: 3 байт)
[20.05.26 21:49:57] Завершение: Запрос успешно обработан
[20.05.26 21:49:58] Новый запрос от 127.0.0.1:37533 (размер: 1 байт)
[20.05.26 21:49:58] Завершение: Ошибка 1 (Размер пакета выходит за рамки 2-256 байт)
[20.05.26 21:49:58] Новый запрос от 127.0.0.1:36182 (размер: 5 байт)
[20.05.26 21:49:58] Завершение: Ошибка 2 (Указанная длина не совпадает с реальным размером данных)
[20.05.26 21:49:58] Новый запрос от 127.0.0.1:36638 (размер: 3 байт)
[20.05.26 21:49:58] Завершение: Ошибка 2 (Указанная длина не совпадает с реальным размером данных)
[20.05.26 21:49:58] Сервер запущен (UDP)
[20.05.26 21:49:59] Новый запрос от 127.0.0.1:42684 (размер: 2 байт)
[20.05.26 21:49:59] Новый запрос от 127.0.0.1:37461 (размер: 2 байт)
[20.05.26 21:50:02] Завершение: Запрос успешно обработан
[20.05.26 21:50:02] Завершение: Запрос успешно обработан
[20.05.26 21:50:02] Сервер запущен (UDP)
[20.05.26 21:50:03] Новый запрос от 127.0.0.1:50349 (размер: 2 байт)
[20.05.26 21:50:03] Завершение: Запрос успешно обработан
[20.05.26 21:50:03] СТАТИСТИКА: Время работы: 7 сек | Успешных: 1 | Ошибок: 3
[20.05.26 21:50:03] СТАТИСТИКА: Время работы: 5 сек | Успешных: 2 | Ошибок: 0
[20.05.26 21:50:03] СТАТИСТИКА: Время работы: 1 сек | Успешных: 1 | Ошибок: 0
[20.05.26 21:50:05] Сервер остановлен
[20.05.26 21:50:05] Сервер остановлен
[20.05.26 21:50:05] Сервер остановлен
```

Рисунок 2

make valgtind_test (рис.3):

```
  ТЕСТИРОВАНИЕ УТЕЧЕК ПАМЯТИ (VALGRIND)
[+] Запуск сервера под Valgrind...
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --trace-children=yes --log-file=valgrind_%p.txt ./lab3asdN3252_server -d -p 8080 -a 1
27.0.0.1
[+] Отправка валидного запроса клиентом (клиент тоже под Valgrind)...
valgrind --leak-check=full --log-file=valgrind_client_%p.txt ./lab3asdN3252_client 127.0.0.1 8080 0x02AABB
Ответ сервера: 10101010 10111011
[+] Отправка невалидного запроса...
./lab3asdN3252_client 127.0.0.1 8080 0x05 || true
Ответ сервера: ERROR 1
[+] Остановка сервера для выгрузки отчетов Valgrind...
pkill -SIGTERM -f "[l]ab3asdN3252_server" || true

  СВОДКА ОШИБОК И УТЕЧЕК VALGRIND
valgrind_1335.txt:==1335== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
valgrind_1336.txt:==1336== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
valgrind_1337.txt:==1337== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
valgrind_1341.txt:==1341== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
valgrind_1345.txt:==1345== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
valgrind_client_1340.txt:==1340== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Подробные отчеты сохранены в файлах valgrind_<PID>.txt
```

Рисунок 3

Журнал сервера (/tmp/lab3.log) продемонстрировал корректную регистрацию всех событий (запуск, входящие пакеты с IP/портами, обработка ошибок протокола и вывод статистики по сигналу SIGUSR1).

Многопроцессная архитектура была проверена на устойчивость: после завершения дочерних обработчиков зомби-процессы в системе не остаются (проверено через ps aux).

ВЫВОД

В ходе выполнения данной лабораторной были реализованы серверная и клиентская программы на языке C для ОС Linux. На практике закреплены механизмы POSIX-систем: управление жизненным циклом процессов (создание рабочих задач через fork, очистка завершенных процессов-зомби с помощью waitpid, демонизация сервера), сетевое взаимодействие на базе UDP-сокетов и перехват асинхронных сигналов. Использование анонимной разделяемой памяти (mmap) в связке с атомарными переменными позволило обеспечить безопасный сбор статистики параллельными процессами без использования блокировок.

ПРИЛОЖЕНИЕ

Исходный код программы server.c:

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <time.h>
#include <fcntl.h>
#include <getopt.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <arpa/inet.h>
#include <stdatomic.h>

// Структура для статистики
typedef struct {
    atomic_ulong success_count;
    atomic_ulong error_count;
    time_t start_time;
} server_stats_t;

// Глобальные переменные
server_stats_t *stats = NULL;
volatile sig_atomic_t keep_running = 1;
volatile sig_atomic_t print_stats_flag = 0;
char *log_file = NULL;
bool debug_mode = false;
```

```
// Функция для записи в лог-файл
void write_log(const char *msg) {
    FILE *f = fopen(log_file, "a");
    if (!f) return;

    time_t t = time(NULL);
    struct tm *tm_info = localtime(&t);
    char time_buf[26];
    strftime(time_buf, sizeof(time_buf), "%d.%m.%y %H:%M:%S", tm_info);

    fprintf(f, "[%s] %s\n", time_buf, msg);
    fclose(f);
}

// Обработчики сигналов
void sig_shutdown(int signum) {
    (void)signum;
    keep_running = 0;
}

void sig_usr1(int signum) {
    (void)signum;
    print_stats_flag = 1;
}

// Обработчик SIGCHLD для уборки зомби-процессов
void sig_chld(int signum) {
    (void)signum;
    int saved_errno = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}
```

```
// Вывод статистики
void print_statistics() {
    time_t now = time(NULL);
    double uptime = difftime(now, stats->start_time);

    char msg[256];
     snprintf(msg, sizeof(msg), "СТАТИСТИКА: Время работы: %.0f сек | Успешных: %lu | Ошибок: %lu", uptime, atomic_load(&stats->success_count), atomic_load(&stats->error_count));

    // Вывод в лог и в stderr
    write_log(msg);
    fprintf(stderr, "%s\n", msg);
}

// Функция демонизации
void daemonize() {
    pid_t pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS); // Родитель завершается

    if (setsid() < 0) exit(EXIT_FAILURE); // Становимся лидером сессии

    pid = fork(); // Второй форк для отвязки от терминала
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);

    chdir("/"); // Переходим в корень

     // Закрываем stdin и stdout, но stderr оставляем для вывода статистики
    int fd = open("/dev/null", O_RDWR);
    if (fd != -1) {
        dup2(fd, STDIN_FILENO);
```

```
        dup2(fd, STDOUT_FILENO);
        if (fd > 2) close(fd);
    }
}

// Обработка данных
void handle_request(uint8_t *buf, ssize_t len, int sock, struct sockaddr_in *cli_addr, socklen_t cli_len, int wait_time) {
    char log_msg[512];
    snprintf(log_msg, sizeof(log_msg), "Новый запрос от %s:%d (размер: %zd байт)", inet_ntoa(cli_addr->sin_addr), ntohs(cli_addr->sin_port), len);
    write_log(log_msg);

    // Имитация долгой работы (-w)
    if (wait_time > 0) sleep(wait_time);

    if (len < 2 || len > 256) {
        atomic_fetch_add(&stats->error_count, 1);
          sendto(sock, "ERROR 1\n", 8, 0, (struct sockaddr*)cli_addr, cli_len);
        write_log("Завершение: Ошибка 1 (Размер пакета выходит за рамки 2-256 байт)");
        return;
    }

    // Совпадение заявленной длины и реального размера пакета
    uint8_t data_len = buf[0];
    if (len != (ssize_t)(data_len + 1)) {
        atomic_fetch_add(&stats->error_count, 1);
          sendto(sock, "ERROR 2\n", 8, 0, (struct sockaddr*)cli_addr, cli_len);
        write_log("Завершение: Ошибка 2 (Указанная длина не совпадает с реальным размером данных)");
```

```
        return;
    }

    // Формирование ответа
    size_t out_len = data_len * 9;
    char *out_buf = malloc(out_len + 1);
    if (!out_buf) {
        atomic_fetch_add(&stats->error_count, 1);
         sendto(sock, "ERROR 3\n", 8, 0, (struct sockaddr*)cli_addr,
cli_len);
        write_log("Завершение: Ошибка 3 (Ошибка выделения памяти)");
        return;
    }

    for (int i = 0; i < data_len; i++) {
        uint8_t b = buf[i + 1];
        for (int bit = 7; bit >= 0; bit--) {
            out_buf[i * 9 + (7 - bit)] = (b & (1 << bit)) ? '1' : '0';
        }
        out_buf[i * 9 + 8] = (i == data_len - 1) ? '\n' : ' ';
    }
    out_buf[out_len] = '\0';

     sendto(sock, out_buf, out_len, 0, (struct sockaddr*)cli_addr,
cli_len);
    free(out_buf);

    atomic_fetch_add(&stats->success_count, 1);
    write_log("Завершение: Запрос успешно обработан");
}

int main(int argc, char *argv[]) {
    // Дефолтные настройки
    int port = 8080;
```

```
    char *ip_addr = "127.0.0.1";
    int wait_time = 0;
    bool is_daemon = false;
    log_file = "/tmp/lab3.log";
    debug_mode = (getenv("LAB3DEBUG") != NULL);

    // Считывание переменных окружения
    if (getenv("lab3WAIT")) wait_time = atoi(getenv("lab3WAIT"));
    if (getenv("lab3LOGFILE")) log_file = getenv("lab3LOGFILE");
    if (getenv("lab3ADDR")) ip_addr = getenv("lab3ADDR");
    if (getenv("lab3PORT")) port = atoi(getenv("lab3PORT"));

    // Парсинг опций
    int opt;
    while ((opt = getopt(argc, argv, "w:dl:a:p:vh")) != -1) {
        switch (opt) {
            case 'w': wait_time = atoi(optarg); break;
            case 'd': is_daemon = true; break;
            case 'l': log_file = optarg; break;
            case 'a': ip_addr = optarg; break;
            case 'p': port = atoi(optarg); break;
             case 'v': printf("Версия: 1.0\nВыполнил: Свиржевский А.Д.,
N3252\nВариант 16\n"); return 0;
            case 'h':
                printf("Опции: -w <сек>, -d (демон), -l <лог>, -a <IP>,
-p <порт>, -v, -h\n");
                return 0;
            default: return EXIT_FAILURE;
        }
    }

    if (is_daemon) daemonize();

    // Создаем разделяемую память для счетчиков
```

```
    stats = mmap(NULL, sizeof(server_stats_t), PROT_READ | PROT_WRITE,
MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (stats == MAP_FAILED) {
        perror("mmap");
        return EXIT_FAILURE;
    }
    stats->success_count = 0;
    stats->error_count = 0;
    stats->start_time = time(NULL);

    // Настройка обработки сигналов
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sig_shutdown;
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGQUIT, &sa, NULL);

    sa.sa_handler = sig_usr1;
    sigaction(SIGUSR1, &sa, NULL);

    sa.sa_handler = sig_chld;
    sa.sa_flags = SA_RESTART; // Чтобы не прерывать recvfrom, если умер
дочерний процесс
    sigaction(SIGCHLD, &sa, NULL);

    // Создание UDP сокета
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        return EXIT_FAILURE;
    }

    struct sockaddr_in serv_addr;
```

```
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr(ip_addr);
    serv_addr.sin_port = htons(port);

    // Позволяет запускать несколько экземпляров
    int optval = 1;
    setsockopt(sock, SOL_SOCKET, SO_REUSEPORT, &optval, sizeof(optval));

     if (bind(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) <
0) {
        perror("bind");
        return EXIT_FAILURE;
    }

    write_log("Сервер запущен (UDP)");

    // Главный цикл сервера
    while (keep_running) {
        if (print_stats_flag) {
            print_statistics();
            print_stats_flag = 0;
        }

        uint8_t buf[1024];
        struct sockaddr_in cli_addr;
        socklen_t cli_len = sizeof(cli_addr);

           ssize_t  n  =  recvfrom(sock,  buf,  sizeof(buf),  0,  (struct
sockaddr*)&cli_addr, &cli_len);
        if (n < 0) {
            if (errno == EINTR) continue; // Прервано сигналом
            perror("recvfrom");
            continue;
```

```
        }

        // Многопроцессность
        pid_t pid = fork();
        if (pid == 0) {
            // Дочерний процесс
           handle_request(buf, n, sock, &cli_addr, cli_len, wait_time);
            exit(EXIT_SUCCESS); // Умирает после обработки
        } else if (pid < 0) {
            perror("fork");
        }
    }

    write_log("Сервер остановлен");
    close(sock);
    munmap(stats, sizeof(server_stats_t));
    return 0;
}
```

Исходный код программы client.c:

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <getopt.h>

void print_help() {
```

```
     printf("Использование: ./lab3asdN3252_client [опции] <IP> <Порт>
<Данные(Hex)>\n");
    printf("Пример: ./lab3asdN3252_client 127.0.0.1 8080 0x02AABB\n");
    printf("  0x02 - длина данных (2 байта)\n");
    printf("  AABB - сами данные\n");
}

int main(int argc, char *argv[]) {
    int opt;
    while ((opt = getopt(argc, argv, "vh")) != -1) {
        if (opt == 'v') {
            printf("Клиент версии 1.0\nСвиржевский А.Д., N3252\nВариант
16\n");
            return 0;
        }
        if (opt == 'h') {
            print_help();
            return 0;
        }
    }

    if (optind + 3 > argc) {
        fprintf(stderr, "Ошибка: недостаточно аргументов\n");
        print_help();
        return EXIT_FAILURE;
    }

    char *ip = argv[optind];
    int port = atoi(argv[optind+1]);
    char *hex = argv[optind+2];

    // Парсинг Hex-строки
    if (strncmp(hex, "0x", 2) == 0 || strncmp(hex, "0X", 2) == 0) hex
+= 2;
```

```
    size_t len = strlen(hex) / 2;
    uint8_t *buf = malloc(len);
    for (size_t i = 0; i < len; i++) {
        char byte_str[3] = {hex[2*i], hex[2*i+1], '\0'};
        buf[i] = (uint8_t)strtol(byte_str, NULL, 16);
    }

    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        return EXIT_FAILURE;
    }

    // Установка таймаута, чтобы клиент не висел вечно
    struct timeval tv = {.tv_sec = 5, .tv_usec = 0};
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    struct sockaddr_in serv_addr;
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);
    inet_pton(AF_INET, ip, &serv_addr.sin_addr);

    // Отправка (UDP)
       sendto(sock,  buf,  len,  0,  (struct  sockaddr*)&serv_addr,
sizeof(serv_addr));
    free(buf);

    // Получение ответа
    char resp[1024];
    ssize_t n = recvfrom(sock, resp, sizeof(resp) - 1, 0, NULL, NULL);
    if (n < 0) {
        fprintf(stderr, "Ошибка получения ответа (таймаут?)\n");
        close(sock);
```

```
        return EXIT_FAILURE;
    }

    resp[n] = '\0';
    printf("Ответ сервера: %s", resp);

    close(sock);
    return 0;
}
```